Parametric Maximum Closure on Directed Forests

Computational Report – data release `v0.3.0`

Valerio Dose Fabio Furini Marco Locatelli

1 September 2026

This report accompanies the manuscript *"On parametric Maximum Closure Problems over precedence forests"*. The manuscript states the algorithms, their analysis, and the few experiments needed to support the theory. This report is the complement: the full experimental record, and specifically the material the manuscript does not carry –

- absolute running times of *every* algorithm at *every* size of every campaign, not a selection;
- breakdowns by *arc density* and by *coefficient family*, which is where the interesting structure turns out to be (Section 8.2);
- what the instance generators actually produced – arcs, components, number of closure layers – so that a reader can tell what each parameter buys (Section 5);
- the dispersion of the raw measurements, so that any reported ratio can be compared against the noise it rests on (Section 13);
- the implementation analysis behind the $\mathcal{O}(n)$ space bound (Section 11), and the exact protocol, including what "peak RSS" means and why memory is only quoted from single-algorithm runs.

Every table and plot is generated from the raw benchmark data by `tools/build_report.sh`; no number is hand-typed. Each figure shows the numbers and the same data plotted, with a comment on how to read them.

Contents

1 Summary

- **Sweep.** One frozen commit, one machine, 2026-08-31/09-01: 16 200 instances, 292 080 timed runs, seven algorithms plus the external `BPPF` baseline. No run hit the 300 s timeout or the 8 GiB memory ceiling, so nothing is censored.
- **Correctness.** Zero cross-algorithm disagreements (Section 6), on top of an exhaustive enumeration oracle on small instances and sanitizer builds.
- **Random forests, paths, binary trees.** `HPaC` is the fastest of our algorithms: 2–13× `RaC` depending on size and density, and faster than the direct scan `PaC` from $n \approx 400$ on.
- **Stars.** The ordering inverts: `RaC` is $\approx 250\times$ `HPaC` at $n = 20\,000$. Any heap-based schedule pays $\Theta(n)$ maintenance per event on a hub; `PaC`, which maintains no candidate structure, resists an order of magnitude better than `HPaC`.
- **Single orientations.** `HIPaC`/`HOPaC` take 0.52–0.78 of `HPaC`'s time, stable up to $n = 10^5$.
- **Against pseudoflow.** `HPaC` is faster than `BPPF` by a median factor 3.3 on forests with $n \le 1\,000$, even with the breakpoints handed to `BPPF` for free; `BPPF` misses the exact layer sequence on 16 of 2 400 instances, for precision reasons (Section 12).
- **Two facts that organize everything else.** The *topology* decides which algorithm wins – density moves the margin by an order of magnitude and the star class inverts it outright – while the *coefficient family* decides only how much parametric structure exists, moving absolute times but never the ranking. And the data structure inside one algorithm can matter as much as the choice of algorithm: the two heap policies of Section 11 differ by 4× in time and two orders of magnitude in memory, at identical output and identical asymptotic bounds.

2 The algorithms, in one page

All of them solve the same problem and return the same object: given a directed forest with integer profits p_i and strictly positive integer weights w_i, the ordered sequence of closure layers of $u(\lambda) = \max\{P(S) - \lambda W(S) : S \text{ closed}\}$, with the exact rational threshold of each layer. All of ours use exact integer arithmetic: 64-bit coefficients, 128-bit cross-multiplication for every ratio comparison, no floating point in any decision path.

Two remarks that matter for reading the results. First, `PaC` and `HPaC` are the *same algorithm* under two schedules – the difference measured in Section 8.1 is entirely the cost of finding the maximum-ratio candidate. Second, `RaC` pays a structural overhead per component (cluster bookkeeping) that the others do not, which is why density matters so much in Section 8.2: a sparse forest is many small components.

Table 1: The algorithms compared in this report. "Time" and "space" are worst-case bounds from the manuscript; the last column is what the experiments below add.

	Applies to	Time	Space	Idea, and what we observe
`PaC`	any forest	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Peel the current best final vertices, or contract the best arc; the best candidate is found by a linear scan at every iteration. Slowest on random forests beyond $n \approx 400$, but the most robust on stars.
`DPaC`	any forest	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	Dual of `PaC`: builds the sequence from the lowest ratio upwards. Tracks `PaC` everywhere.
`HPaC`	any forest	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n)$	Same two moves, but the candidates live in two heaps, so only those touched by the last move are refreshed. Fastest of ours on random forests, paths and binary trees; penalized on a high-degree hub.
`DHPaC`	any forest	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n)$	Dual of `HPaC`; within 25% of it everywhere.
`HIPaC`	in-forests	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	When every vertex has out-degree ≤ 1 the minimal preceding set of an arc is a singleton, so one heap value per vertex suffices and no closure sums are propagated. Takes 0.52–0.75 of `HPaC`'s time.
`HOPaC`	out-forests	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	The symmetric case (in-degree $\leq$ 1): 0.57–0.78 of `HPaC`.
`RaC`	any tree	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	Rake-and-compress on a top tree: contracts the tree in $\mathcal{O}(\log n)$ rounds computing cluster functions, then recovers the thresholds top-down. The only algorithm whose cost does not depend on how often a single vertex is touched – hence the winner on stars, by up to 250×.
`BPPF`	general graphs	$\mathcal{O}(mn \log n)$	$\mathcal{O}(m)$	External baseline: bounded-precision parametric pseudoflow (unmodified upstream), solves a strictly more general problem. The bound is for the whole parametric sweep on a graph with m arcs; on a forest $m = \mathcal{O}(n)$, so it reads $\mathcal{O}(n^2 \log n)$ – worse than every algorithm above, which is the theoretical reason a forest-specific method is worth having. Evaluates minimum cuts at parameter values supplied by the caller and uses fixed-point arithmetic.

3 Definitions and conventions

Terms used throughout, fixed here once.

- **Closure layer.** One block $\mathcal{I}_r$ of the canonical partition of the vertex set induced by the parametric solution: the vertices that enter the optimal closed set exactly when λ crosses the breakpoint λ_r. "Number of layers" equals the number of breakpoints of $u(\lambda)$, so it measures how much parametric structure an instance has, and it bounds the number of iterations of every algorithm here.
- **Threshold** of a vertex: the breakpoint at which that vertex enters the optimal closed set; vertices with equal threshold form one layer.
- **Density ϱ.** The probability with which the generator adds each candidate arc. A forest with density ϱ has $\varrho(n-1)$ arcs and $n-\varrho(n-1)$ connected components in expectation, so $\varrho = 1.0$ gives one spanning tree and $\varrho = 0.3$ a forest of many small trees (Fig. 1).
- **Coefficient family.** The rule used to draw the profits and weights of an instance; six are used (Section 5). It controls the *shape* of the ratios p_i/w_i and hence how many layers appear.
- **Paired ratio.** The ratio of two algorithms' times *on the same instance*. Reported as the median over instances, never as a ratio of two aggregates: pairing removes the instance-to-instance variation that would otherwise swamp the comparison.
- **IQR (interquartile range).** $Q_3 - Q_1$: the width of the band containing the central half of the observations. It is to the median what the standard deviation is to the mean, but insensitive to the tails. Beside a median ratio, a large IQR signals that the population mixes regimes instead of scattering around one value – exactly what happens across densities in Section 8.2.
- **Relative IQR** (Section 13): the IQR of the repetitions of one instance divided by that instance's median time, in percent. It quantifies measurement noise, not algorithmic variability.
- **Peak RSS** (resident set size): the maximum amount of physical RAM the process had mapped during the run, from `getrusage(RUSAGE_SELF).ru_maxrss`. See Section 4 for the two caveats that make it usable only for single-algorithm runs.
- **Censored run.** A run killed by the time limit (300 s) or the memory ceiling (8 GiB), hence without a measured time. There are none in this study; the term appears only to state that.
- **Campaign.** One preregistered block of the sweep – a set of instances plus the algorithms run on them (B, C, D, E, G).

4 Setup

Machine and build. Intel Core i7-12700, 32 GB RAM, Ubuntu 22.04.5 (kernel 6.8), GCC 11.4.0 at `-O3`, single-threaded. Each benchmark process pinned to one physical core (`taskset`); the six campaign lanes ran on distinct cores, never two on the same core. CPU governor `powersave` (no passwordless root on this machine), so absolute times carry ordinary frequency-scaling noise – which is one reason every comparison is a paired same-instance ratio.

What is timed. Only the algorithm call (`std::chrono::steady_clock`); parsing, result hashing and correctness comparison sit outside the timed region. Repetitions: 11 for random forests with $n \leq 1\,000$, 3 elsewhere. Algorithm order shuffled per repetition, so no algorithm is systematically favoured by the CPU frequency state.

Memory, and what "peak RSS" means. RSS is the *resident set size*: the amount of physical RAM the process has mapped at a given instant. Peak RSS is its maximum over the run, read from `getrusage(RUSAGE_SELF).ru_maxrss`. Two properties matter when reading memory figures. It is a *process-lifetime* peak and never decreases, so in an invocation that benchmarks several algorithms

the value recorded for the second includes the peak of the first – memory is therefore quoted only from single-algorithm invocations (Section 11). And it counts pages actually resident, so it includes allocator overhead and is precisely the quantity that decides whether a run survives a memory ceiling.

Statistics. Absolute times: median over repetitions, then median over instances. Comparisons: *paired* per instance, reported as the median of the per-instance ratios together with their IQR (Section 3), never a ratio of aggregate means. The median is chosen because it is invariant under inversion of the comparison (the arithmetic mean of ratios is not: it depends on which algorithm sits in the denominator), because the timing noise has an asymmetric right tail by construction (interference can only slow a run down, never speed it up), and because it is the conservative choice on every comparison where we claim an advantage – on the `BPPF` comparison at $n = 1\,000$ the three aggregates are 4.5 (median), 5.8 (geometric mean) and 8.7 (arithmetic mean).

5 Instances

An instance of this problem is two independent things: a *precedence structure* (which vertex forces which) and a set of *affine coefficients* (a profit and a weight per vertex). We generate the two separately and cross them, so that any effect observed can be attributed to one or the other. Concretely, one instance is fully determined by four choices: a topology, a coefficient family, a size n, and a random seed. Nothing else is tuned – in particular there is no capacity, no time horizon and no scaling of the coefficients.

Files use the `.pcf` format: the arc list, one integer profit and one strictly positive integer weight per vertex, where arc (u, v) encodes the implication $x_u \leq x_v$. The format has no capacity field and no reader accepts one – a deliberate choice, since the same algorithms exist in the literature for capacitated variants and we did not want the two to be confusable. Every instance is written by a seeded deterministic Python generator, never by hand, and is recorded in a manifest with its SHA-256 checksum, its topology classification and its coefficient bounds; a regenerated or downloaded archive is therefore verifiable byte-for-byte against the one used here.

Below, each of the two components is described with the rule that generates it. Section 5.1 then measures what the combination actually produces – how many closure layers, and how big – which is the quantity that governs how much work every algorithm has to do.

Coefficient families (six).

- `independent-positive`: w_i, p_i independent uniform on $\{1, \ldots, 1000\}$.
- `independent-signed`: w_i as above, p_i uniform on $\{-1000, \ldots, 1000\}$.
- `correlated`: $p_i = w_i + \delta_i$, δ_i uniform on $\{-50, \ldots, 50\}$.
- `anti-correlated`: $p_i = (1001 - w_i) + \delta_i$.
- `near-ties`: one target ratio num/den per instance (num, den uniform on $\{1, \ldots, 20\}$), then t_i uniform on $\{1, \ldots, 50\}$, $w_i = \mathrm{den}\, t_i$, $p_i = \mathrm{num}\, t_i + \jmath_i$ with jitter $\jmath_i \in \{-2, -1, 1, 2\}$.
- `exact-ties`: $g = \max(1, \lfloor n/20 \rfloor)$ groups each with its own exact ratio; vertex i goes to group $i \bmod g$, so equal-threshold vertices are spread over the graph rather than adjacent.

Topologies (six). `mixed-forest`: each $v \geq 2$ linked to a uniform predecessor $u < v$ with probability ϱ, each edge oriented (u, v) or (v, u) with probability 1/2. `in-forest`: one arc (u, j) with j uniform above u, with probability ϱ (out-degree ≤ 1); `out-forest` symmetric (in-degree ≤ 1). `path-mixed`, `binary-mixed`, `star-mixed`: fixed underlying shape (path; balanced binary tree, parent $\lfloor (v-1)/2 \rfloor$; star, hub 0), orientations randomized.

Sizes and counts. 240 instances per topology and size (four densities × six families × ten seeds), 60 for the structured families, which have no density parameter. The random topologies use the ten sizes $100, 200, \ldots, 1\,000$ and the ten sizes $10\,000, 20\,000, \ldots, 100\,000$; the structured families use 100, 200, 500, 1 000, 2 000, 5 000, 10 000, 20 000, 50 000 and 100 000.

	median over campaign C		
ϱ	arcs	components	closure layers
0.3	16370	38476	23234
0.6	32930	21944	19845
0.9	49477	5498	15943
1.0	54999	1	14718

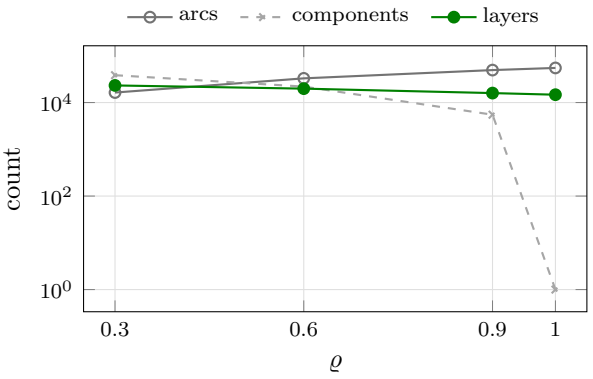

Figure 1: What the density parameter actually produces (campaign C, medians). Arcs grow linearly in ϱ and components fall accordingly, from a forest of $\approx 38\,500$ trees at $\varrho = 0.3$ to a single spanning tree at $\varrho = 1.0$. The number of closure layers moves the other way and far less: connecting the forest removes only about a third of them, because precedence constraints merge layers that would otherwise be distinct. This is why density is a genuine experimental factor and not a cosmetic one.

	median # closure layers	
family	$n \leq 1\,000$	$n \geq 10\,000$
`anti-correlated`	342	30379
`correlated`	337	24825
`exact-ties`	105	7884
`independent-positive`	344	32374
`independent-signed`	346	33183
`near-ties`	164	1222

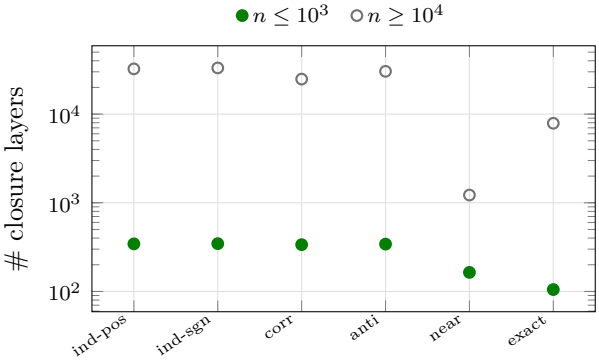

Figure 2: How much parametric structure each coefficient family creates. This is the single most useful fact for reading absolute times, since the work of every algorithm scales with the number of breakpoints: at $n \geq 10^4$ the median layer count ranges from 1 222 (`near-ties`) to 33 183 (`independent-signed`), a factor of 27 at equal size. It also explains why the tie-stressing families are the *fastest* in every timing table below, which would otherwise look counter-intuitive.

5.1 How big are the closure layers?

The campaign CSVs record how *many* layers an instance has, not how big they are: the algorithms are timed and their output is discarded. The size distribution is worth one measurement of its own, because it is what a peel step actually consumes. Figure 4 reports it on a sample of 60 instances at $n = 10\,000$ (all six families, four densities, plus stars, paths and in-forests), 261 232 layers in total.

	# closure layers		
n	median	min	max
100	57	10	91
200	106	24	177
300	158	35	267
400	205	53	350
500	265	66	438
600	314	80	521
700	366	95	611
800	418	107	709
900	470	124	782
1 000	528	134	873
10 000	5032	436	8449
20 000	9826	558	16728
30 000	14718	645	24852
40 000	19434	674	32779
50 000	24226	741	40675
60 000	28663	787	48463
70 000	32662	825	55961
80 000	36690	864	63666
90 000	40251	888	71224
100 000	43390	915	78393

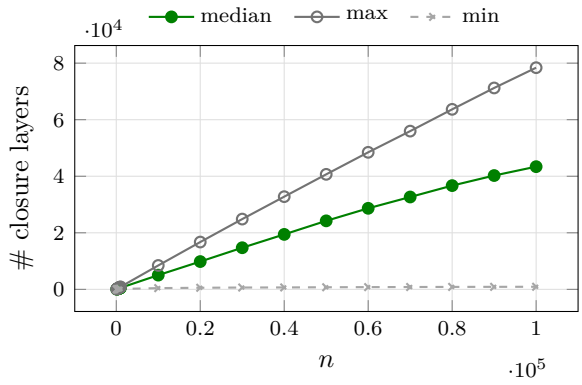

Figure 3: Closure layers by size over both random campaigns, with the observed spread. The layer-to-vertex ratio settles around 0.15 at $n = 10^5$: a 10^5-vertex instance typically has of the order of 15 000 breakpoints to enumerate, and the spread between the easiest and hardest instance of one size is roughly a factor of three.

group	layers	median	mean	max	singletons (%)
`anti-correlated`	60402	1	1.66	5029	73.1
`correlated`	56228	1	1.78	4997	68.2
`exact-ties`	16072	3	6.22	4960	0.0
`independent-positive`	61107	1	1.64	5065	74.4
`independent-signed`	61520	1	1.63	5077	75.3
`near-ties`	5903	7	16.94	4930	0.0
`gen`, $\varrho = 0.3$	33866	1	1.77	563	78.3
`gen`, $\varrho = 0.6$	29163	1	2.06	740	67.8
`gen`, $\varrho = 0.9$	23724	1	2.53	835	59.4
`gen`, $\varrho = 1.0$	22272	1	2.69	778	58.1
`in`, $\varrho = 0.3$	34065	1	1.76	522	78.8
`in`, $\varrho = 0.6$	29351	1	2.04	655	67.6
`in`, $\varrho = 0.9$	24232	1	2.48	803	58.2
`in`, $\varrho = 1.0$	22239	1	2.70	859	55.2
`path`	22885	2	2.62	1111	41.5
`star`	19435	1	3.09	5077	95.1

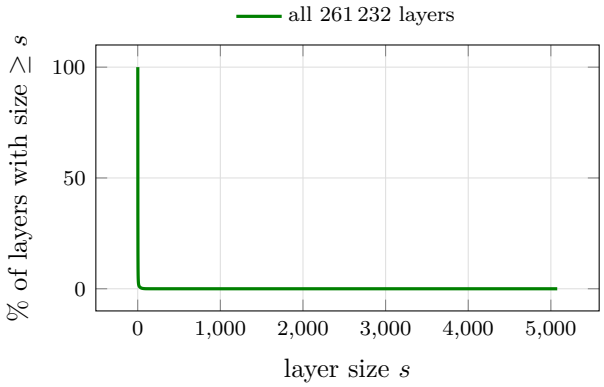

Figure 4: Size of the closure layers, measured on 60 instances at $n = 10\,000$ (261 232 layers). *The distribution is extremely skewed*: overall median 1, mean 2.3, and two thirds of all layers are singletons – yet the largest single layer holds 5 077 vertices, half the graph. So a parametric solution is typically a long chain of one-vertex layers plus a handful of very large blocks, and an algorithm's cost per layer matters more than its cost per vertex. Two group-level facts stand out. The tie-stressing families behave completely differently from the others: `near-ties` has median layer size 7 and *no* singletons at all (by construction, vertices share thresholds), against median 1 and 73–75% singletons for the independent families – which is the mechanism behind their much smaller layer counts (Fig. 2). And density compresses the largest layer by an order of magnitude: on sparse random forests the biggest layer holds 563 vertices, against $\approx 5\,000$ once the graph is a single tree, because a layer cannot straddle components.

6 Validation

- **Exhaustive enumeration oracle** (CTest): every directed forest with ≤ 4 vertices over a finite coefficient grid; 10 000 random forests up to 11 vertices; mixed-orientation path, binary and star trees up to 257 vertices; 6 000 in-forest and 6 000 out-forest instances plus 2 000 larger ones per orientation; plus the structural invariants (layers partition V, each is a closure increment, thresholds strictly decreasing).
- **Cross-algorithm differential agreement**: every campaign runs several algorithms on the same instance and compares partition, thresholds and canonical order through a 64-bit fingerprint. Over the sweep: *zero* disagreements (Table 2).
- **Sanitizers and CI**: GCC Release, GCC Debug with the address and undefined-behaviour sanitizers, and Clang Debug; all three pass at the frozen commit (`results/TEST_REPORT_2026-08-31.md`).
- **Overflow**: `validate_instance` refuses any instance whose total absolute profit or weight would leave the exact-arithmetic margin (`INT64_MAX`/4), before any algorithm runs.

Table 2: Instances benchmarked and cross-algorithm disagreements per campaign. Campaign G (Section 12) is listed apart, `BPPF` being an external baseline rather than one of our algorithms.

Campaign	Instances	Mismatches
campaign_b	2400	0
campaign_c	2400	0
campaign_d_binary	600	0
campaign_d_path	600	0
campaign_d_star	600	0
campaign_e_in	4800	0
campaign_e_out	4800	0

7 The campaigns

A *campaign* is one block of the sweep: a set of instances, the algorithms run on them, and the number of repetitions. The split is not cosmetic – each campaign answers one question, and keeping them separate is what allows a result to be attributed to a cause:

- **A** is correctness only, on instances small enough to be solved by brute force, so that every algorithm can be checked against an independent oracle rather than against another algorithm.
- **B** and **C** are the same random topology at two scales. B is where all five algorithms still fit, so it is the one place where every pairwise comparison is available; C drops the two quadratic ones and asks how the rest scale.
- **D** fixes the shape (path, balanced binary tree, star) and varies nothing but orientations and coefficients, which isolates the effect of topology – and produces the one case where the theoretical separation between our algorithms becomes visible.
- **E** restricts the orientation (in-forest, out-forest) to measure what a specialized algorithm gains from knowing it in advance.
- **G** is the external comparison against parametric pseudoflow. It sits on the same instances as B, so its numbers can be read against the rest of the study.

All campaigns were frozen in `docs/EXPERIMENTAL_PLAN_V3.md` before any run, together with the two size cutoffs, so that no scoping decision was taken after seeing results. Table 3 gives the definitions, Table 4 how much work each algorithm actually did, and Table 2 the correctness outcome per

campaign.

Table 3: Campaign definitions. Every campaign uses all six coefficient families and ten seeds; the random topologies use all four densities. Two preregistered size cutoffs apply, in both cases because the asymptotic trend is already established below them and running further would consume hours without adding information: `PaC`/`DPaC` stop at $n = 20\,000$ on random forests, and `HPaC`/`DHPaC` stop at $n = 20\,000$ on stars (where `PaC` is cheap and runs the full range instead).

	Topology	Sizes n	Inst./size	Reps	Algorithms
A	all six	≤ 11 (exhaustive at 4)	–	–	all, against the enumeration oracle
B	`mixed-forest`	$100, 200, \ldots, 1\,000$	240	11	`PaC`, `DPaC`, `HPaC`, `DHPaC`, `RaC`
C	`mixed-forest`	$10\,000, \ldots, 100\,000$	240	3	`HPaC`, `DHPaC`, `RaC`; `PaC`, `DPaC` at 10^4, $2 \cdot 10^4$
D	`path-`, `binary-`, `star-mixed`	10 sizes, $100 \ldots 100\,000$	60	3	`HPaC`, `RaC`, `DHPaC`, `PaC`, `DPaC`
E	`in-`, `out-forest`	20 sizes, $100 \ldots 100\,000$	240	3	`HPaC`, `DHPaC`, `HIPaC`/`HOPaC`, `RaC`
G	`mixed-forest`	$100, 200, \ldots, 1\,000$	240	3	`HPaC` against `BPPF`

Table 4: Work actually done per algorithm over the sweep. The runs-per-instance column reflects the mix of repetition counts and cutoffs: `PaC` and `DPaC` appear on fewer instances (size cutoffs) but with more repetitions each, since they were mostly exercised on the medium campaign where 11 repetitions were used. `RaC` appears on the most instances, being the only algorithm run on every class at every size.

algorithm	instances	timed runs	runs/instance
`PaC`	4 080	31 200	7.6
`DPaC`	3 480	29 400	8.4
`HPaC`	16 080	67 440	4.2
`DHPaC`	16 080	67 440	4.2
`HIPaC`	4 800	14 400	3.0
`HOPaC`	4 800	14 400	3.0
`RaC`	16 200	67 800	4.2
total	16 200	292 080	

8 Random forests

The two random campaigns answer two different questions. At medium sizes the question is whether the heap is worth its maintenance cost (Section 8.1); at large sizes it is how the three surviving algorithms scale, and how much the answer depends on the density of the forest (Section 8.2).

8.1 Medium sizes: direct scan versus heap

Campaign B runs all five of our algorithms on the same 2 400 instances, which makes it the one place where the primal/dual and scan/heap choices can be compared without any confounding factor. It is also the size range where the heap's overhead is still comparable to its benefit, so it is where the crossover can be located.

8.2 Large sizes, and the effect of density

Campaign C drops the two direct-scan algorithms beyond $n = 20\,000$ (their quadratic growth is established by then) and asks the question that matters at scale: how do `HPaC`, its dual and `RaC`

	CPU time (ms)				
n	PaC	DPaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.1	0.3
200	0.2	0.2	0.2	0.2	0.6
300	0.3	0.3	0.3	0.3	0.9
400	0.5	0.5	0.3	0.3	1.2
500	0.7	0.7	0.4	0.4	1.4
600	0.9	1.0	0.5	0.5	1.8
700	1.2	1.3	0.6	0.6	2.0
800	1.8	1.6	0.8	0.7	2.7
900	2.2	2.0	0.9	0.8	3.3
1 000	2.2	2.4	0.9	0.9	3.0

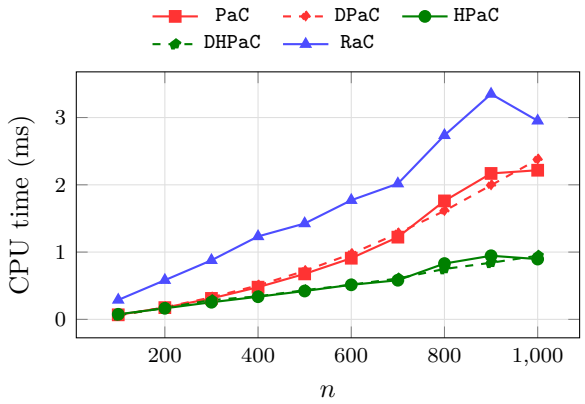

Figure 5: All five algorithms on `mixed-forest` instances, $n \in \{100, 200, \ldots, 1\,000\}$ (240 instances per size). The two families are indistinguishable up to $n \approx 300$ and the heap pays off from $n \approx 400$ on; each dual tracks its primal throughout, which is why the manuscript reports only the primals. `PaC` re-scans every candidate ratio at every iteration, while `HPaC` extracts the maximum in logarithmic time and refreshes only the candidates touched by the last peel or contraction.

n_nodes	Paired instances	Median pac/hpac	IQR
100	240	0.910	0.609
200	240	1.061	0.747
300	240	1.187	0.879
400	240	1.311	1.057
500	240	1.451	1.269
600	240	1.650	1.370
700	240	1.788	1.490
800	240	1.937	1.723
900	240	2.055	1.961
1000	240	2.293	2.133

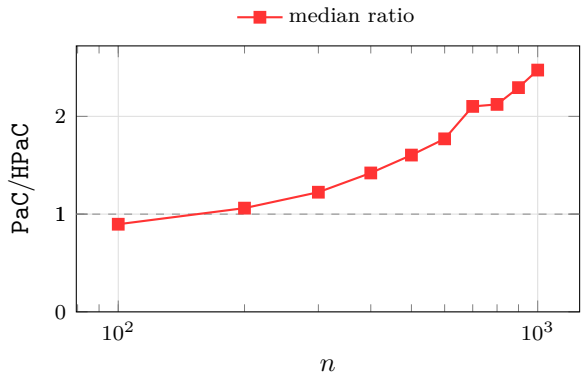

Figure 6: Paired ratio `PaC`/`HPaC` by size, with its IQR: 0.91 at $n = 100$ (the scan is marginally faster where heap maintenance does not yet pay for itself), crossing 1 near $n = 400$, reaching 2.3 at $n = 1\,000$. Beyond this range it keeps growing – 20.3 at $n = 10^4$, 42.9 at $n = 2 \cdot 10^4$ – which is the practical meaning of the $\mathcal{O}(n^2)$ against $\mathcal{O}(n \log n)$-in-practice difference. The dashed line marks parity.

	median CPU time (ms)		
ϱ	PaC	HPaC	RaC
0.3	0.6	0.2	0.6
0.6	0.7	0.3	1.2
0.9	0.8	0.7	2.1
1.0	0.9	0.9	2.3

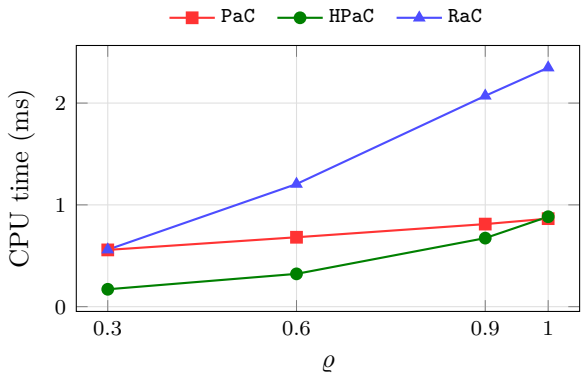

Figure 7: Campaign B by density. Every algorithm slows down as the forest becomes more connected, but not by the same factor: `HPaC` roughly doubles from $\varrho = 0.3$ to $\varrho = 1.0$ while `RaC` grows less. This is the medium-size beginning of the density effect that dominates Section 8.2.

	median CPU time (ms)		
family	PaC	HPaC	RaC
anti-correlated	0.8	0.4	1.2
correlated	0.8	0.4	1.2
exact-ties	0.5	0.3	1.2
independent-positive	0.8	0.4	1.3
independent-signed	0.8	0.4	1.2
near-ties	0.6	0.3	1.2

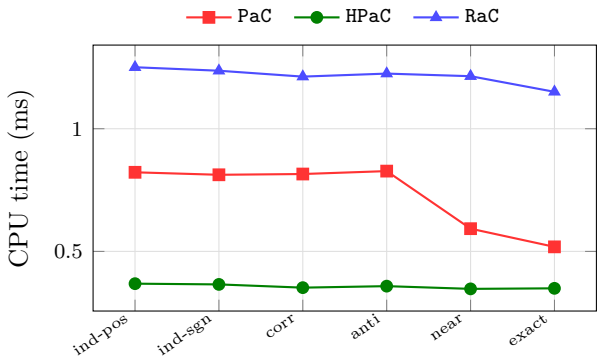

Figure 8: Campaign B by coefficient family. The two tie-stressing families are the fastest for every algorithm, consistently with their smaller layer counts (Fig. 2); the ordering between algorithms is the same in all six, so no family reverses a conclusion.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	3.117	0.674
correlated	400	3.173	0.684
exact-ties	400	3.300	0.724
independent-positive	400	3.187	0.633
independent-signed	400	3.183	0.636
near-ties	400	3.166	0.723

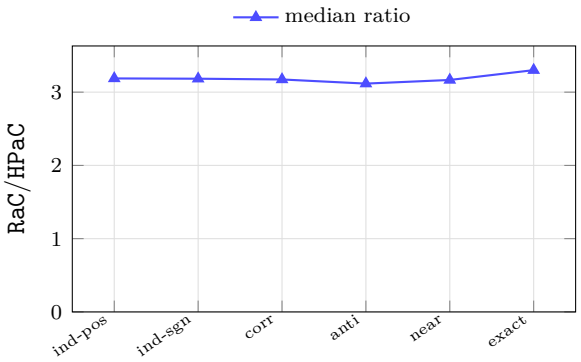

Figure 9: Campaign B, paired ratio RaC/HPaC by family: flat at 3.1–3.3 across all six. At these sizes the coefficient family does not affect which algorithm wins.

compare on forests with up to 10^5 vertices, and how much of the answer depends on the density of the forest. As it turns out, more than it depends on the size.

	CPU time (ms)		
n	`HPaC`	`DHPaC`	`RaC`
10 000	12.7	13.2	50.5
20 000	28.0	29.8	131.5
30 000	44.9	47.2	262.8
40 000	62.9	66.9	399.7
50 000	84.3	92.3	543.9
60 000	96.1	117.9	652.5
70 000	117.4	141.7	837.0
80 000	141.1	161.5	1 027.5
90 000	164.9	198.1	1 304.7
100 000	203.9	214.6	1 709.4

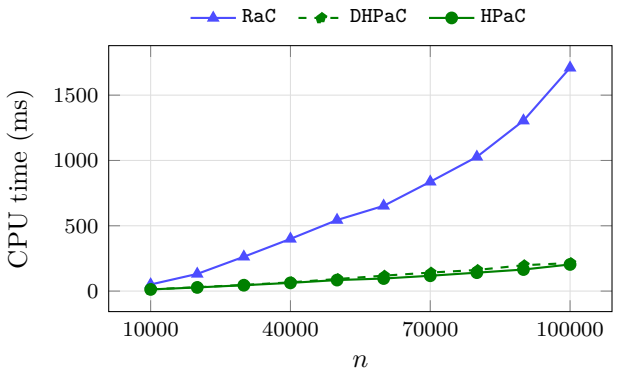

Figure 10: `mixed-forest` instances, $n \in \{10\,000, \ldots, 100\,000\}$ (240 per size). `HPaC` computes the full parametric solution on 10^5 vertices in about 0.2 s; `DHPaC` stays within 25% of it at every size, and is marginally faster at $n = 10^5$.

n_nodes	Paired instances	Median rac/hpac	IQR
10000	240	4.178	2.336
20000	240	5.105	4.290
30000	240	6.132	6.399
40000	240	7.097	8.021
50000	240	7.857	9.819
60000	240	9.207	13.008
70000	240	10.374	15.355
80000	240	11.287	16.886
90000	240	12.472	18.374
100000	240	13.256	19.445

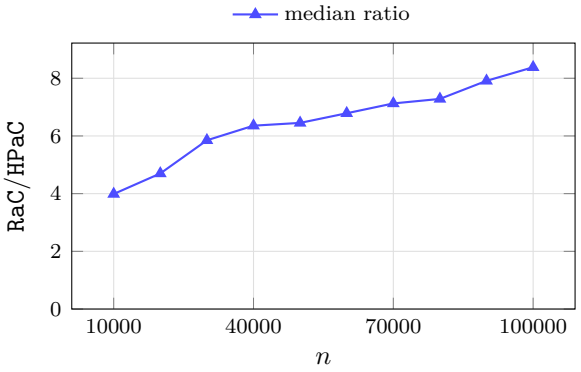

Figure 11: Paired ratio `RaC`/`HPaC` by size, with IQR: from 4.2 at $n = 10^4$ to 13.3 at $n = 10^5$. The IQR widens with n, so the disadvantage grows *and* becomes more variable – the next figure explains why.

8.3 What the random campaigns say

Three things follow from Figs. 5, 6, 10 to 12 and 15 and are worth stating explicitly, because none of them is visible from the complexity bounds alone.

- **The heap is worth it, but only past a size threshold.** Below $n \approx 300$ the direct scan is as good and at $n = 100$ marginally better: the heap's maintenance is not amortized yet. The threshold is small enough that it does not matter in practice, but it does mean that for tiny instances the simpler algorithm is the right choice.
- **Density is a first-order factor, and it acts against `RaC`.** The `RaC`/`HPaC` ratio moves from 2.0 to 17.0 as ϱ goes from 1.0 to 0.6 – a bigger swing than going from $n = 10^4$ to $n = 10^5$ at fixed density. Anyone benchmarking these algorithms on a single density would draw a substantially wrong conclusion about the margin. The reason is structural: `RaC` pays cluster bookkeeping per component, and a sparse forest is thousands of tiny components.
- **The coefficient family is a second-order factor.** It changes absolute times by at most 14% and the paired ratio by less than 6%, even though it changes the number of closure layers by a

rho	Paired instances	Median rac/hpac	IQR
0.3	600	12.924	10.236
0.6	600	17.029	12.518
0.9	600	4.972	1.893
1.0	600	2.004	0.359

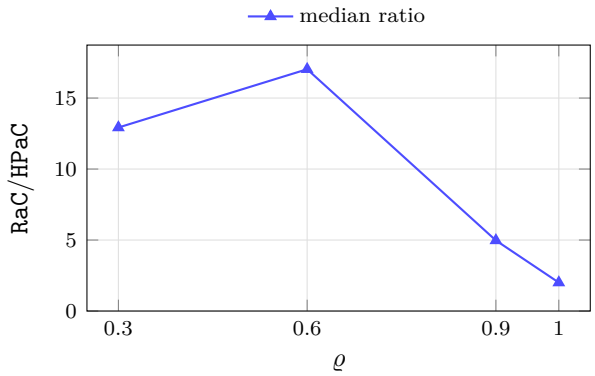

Figure 12: The same pairing aggregated by density, and the reason a single aggregate would mislead: the median ratio is 12.9 at $\varrho = 0.3$, 17.0 at $\varrho = 0.6$, 5.0 at $\varrho = 0.9$ and only 2.0 on single spanning trees, with the IQR shrinking from 10.2 to 0.4 in the same direction. A sparse forest splits into many small components, on which `HPaC`'s contractions are almost free while `RaC` still pays cluster bookkeeping per component; on one spanning tree the two are closest. The ordering never reverses, but the margin varies by almost an order of magnitude – the population is a mixture of four regimes, which is exactly what the widening IQR of Fig. 11 is made of.

	median CPU time (ms)		
ϱ	`HPaC`	`DHPaC`	`RaC`
0.3	27.8	34.4	349.2
0.6	52.4	62.3	902.1
0.9	140.7	159.3	709.4
1.0	221.4	261.4	432.7

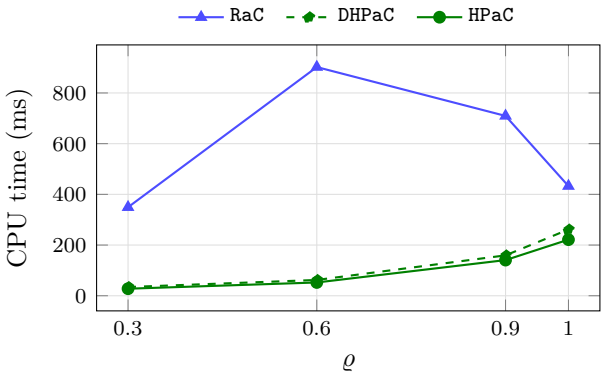

Figure 13: Campaign C by density, absolute times: the gap between `RaC` and the heap-based algorithms closes as the forest becomes connected, and all three get faster – fewer components means fewer, larger closure layers (Fig. 1).

	median CPU time (ms)		
family	`HPaC`	`DHPaC`	`RaC`
`anti-correlated`	68.5	78.2	563.3
`correlated`	67.2	75.6	543.8
`exact-ties`	63.0	71.2	505.1
`independent-positive`	70.7	79.2	559.9
`independent-signed`	70.2	77.8	552.1
`near-ties`	62.0	68.0	490.3

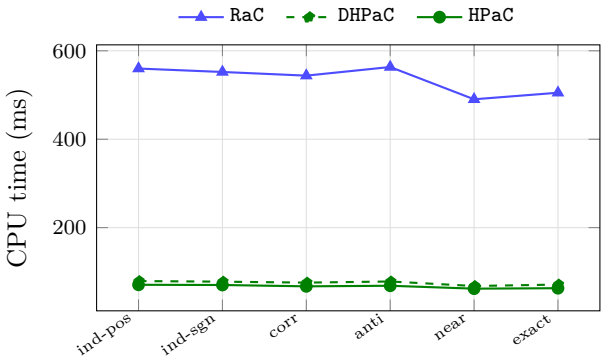

Figure 14: Campaign C by family, absolute times: at most 14% spread per algorithm – an order of magnitude less than the spread across densities.

family	#inst	median RaC/HPaC	IQR
anti-correlated	400	6.002	11.174
correlated	400	6.023	11.491
exact-ties	400	5.983	13.354
independent-positive	400	5.796	11.053
independent-signed	400	5.904	10.993
near-ties	400	6.110	13.377

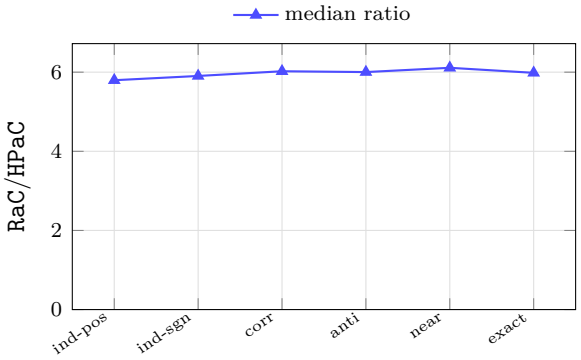

Figure 15: Campaign C, paired ratio by family: between 5.80 and 6.11 in all six. Read against Fig. 12, this is the clearest single statement of the study – the topology decides which algorithm wins, the coefficient family only how much work there is to do.

factor of 27 (Fig. 2). Time per layer, not time per vertex, is what the family moves – and it moves it for all algorithms together.

9 Structured classes

Paths, balanced binary trees and stars share one property that the random forests do not: the topology is fixed, so only the orientations and the coefficients vary. They therefore isolate the effect of the shape, and they are the classes where the theoretical complexity separation becomes visible. Paths and binary trees confirm the random-forest picture; stars invert it.

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.5
200	0.2	0.2	0.2	1.1
500	0.8	0.4	0.4	2.9
1 000	2.9	0.8	0.9	6.1
2 000	10.5	1.7	1.8	12.9
5 000	–	4.4	4.7	37.9
10 000	–	9.0	9.7	80.5
20 000	–	19.2	20.9	175.3
50 000	–	54.9	60.6	475.8
100 000	–	117.6	134.0	926.2

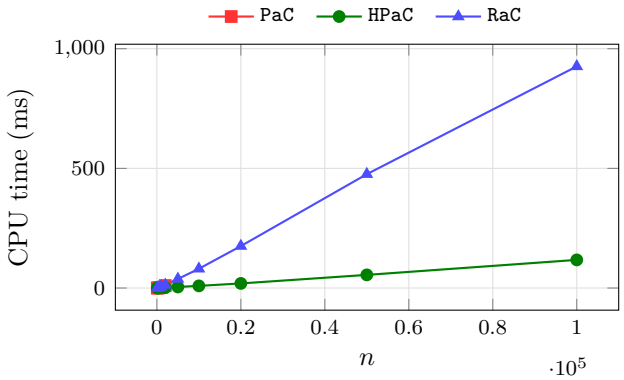

Figure 16: `path-mixed` instances (60 per size). These behave like random forests: median `RaC/HPaC` between 5.8 and 9.0 over the whole range, and `PaC` left far behind by $n = 10^4$, as its $\mathcal{O}(n^2)$ bound predicts.

9.1 What the structured classes say

- **Paths and binary trees add no new regime.** The ratios sit where the random forests put them (5.8–9.0 and 3.2–3.9) and are flat in n over three orders of magnitude: on these shapes the asymptotic advantage of `RaC` never materializes within the sizes we can run.
- **Stars are the counterexample the theory predicts, and they are extreme.** The ratio moves by four orders of magnitude across the size range, from 0.75 to 0.004. This is the one place in the study where the complexity separation is not academic.

	median CPU time (ms)		
family	PaC	HPaC	RaC
anti-correlated	0.9	3.1	25.8
correlated	0.9	3.1	25.3
exact-ties	0.7	2.9	22.3
independent-positive	0.9	3.1	25.8
independent-signed	0.8	3.0	25.5
near-ties	0.7	2.9	21.2

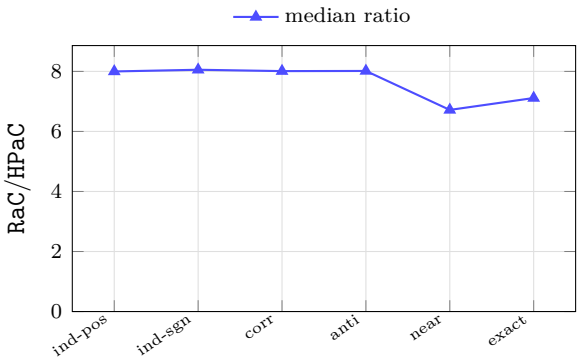

Figure 17: Paths by family: absolute times (left) and the paired ratio RaC/HPaC (right). Flat across families, like the random campaigns.

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.1	0.1	0.4
200	0.2	0.2	0.2	0.9
500	0.9	0.6	0.7	2.3
1 000	3.1	1.3	1.4	4.9
2 000	11.2	2.6	2.7	10.1
5 000	–	6.8	7.2	27.3
10 000	–	14.3	15.0	57.5
20 000	–	32.0	32.1	123.1
50 000	–	92.3	92.5	315.6
100 000	–	205.6	211.1	663.7

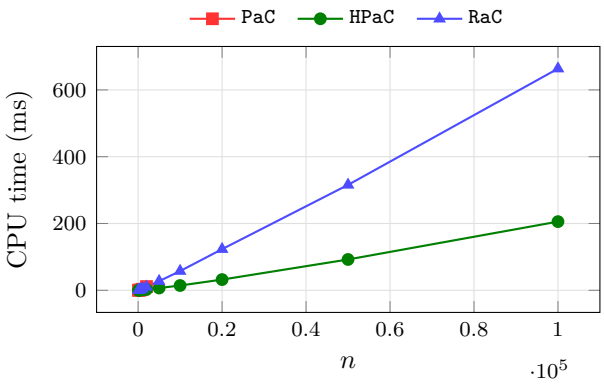

Figure 18: binary-mixed instances (60 per size): median RaC/HPaC between 3.2 and 3.9 over the whole range – lower than on paths, since a balanced tree gives RaC's contraction rounds more to work with.

	median CPU time (ms)		
family	PaC	HPaC	RaC
anti-correlated	0.9	4.7	18.9
correlated	0.9	4.6	18.7
exact-ties	0.7	4.4	16.0
independent-positive	0.9	4.7	18.7
independent-signed	0.9	4.7	18.7
near-ties	0.8	4.5	16.0

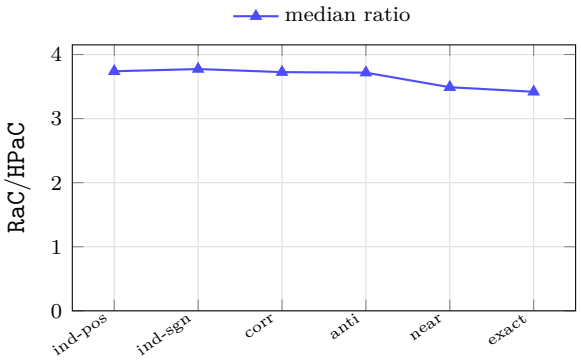

Figure 19: Binary trees by family: absolute times (left), paired ratio (right).

	CPU time (ms)			
n	PaC	HPaC	DHPaC	RaC
100	0.1	0.6	0.6	0.4
200	0.2	2.0	2.0	0.8
500	1.1	12.6	13.2	2.1
1 000	4.5	62.0	62.0	5.2
2 000	17.6	231.2	226.1	9.7
5 000	123.7	1 427.5	1 429.4	24.7
10 000	643.1	6 898.7	6 479.8	63.3
20 000	2 890.8	26 127.1	24 619.1	122.1
50 000	19 336.1	–	–	291.7
100 000	77 541.1	–	–	606.8

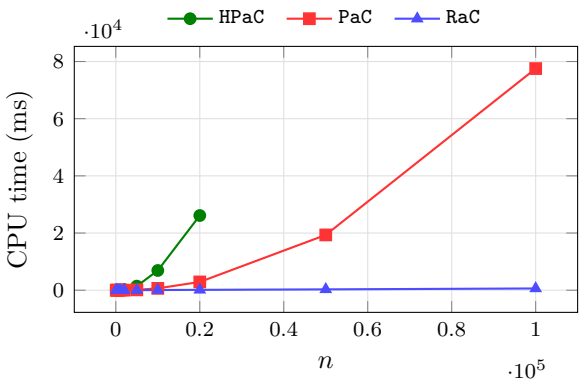

Figure 20: `star-mixed` instances (60 per size) – the class that inverts the ordering. `RaC` is already faster at $n = 100$ and $\approx 250\times$ faster at $n = 20\,000$. Every peel or contraction touches the hub and invalidates the candidate ratios of a constant fraction of the remaining leaves, so *any* heap-based schedule pays $\Theta(n)$ maintenance per event, while `RaC`'s rake operations absorb all leaves in $\mathcal{O}(\log n)$ rounds regardless of the hub's update history. `PaC`, which maintains no candidate structure at all, stays an order of magnitude below `HPaC`. The heap-based algorithms stop at the preregistered cutoff $n = 20\,000$, where their quadratic trend is established.

	median CPU time (ms)		
family	PaC	HPaC	RaC
`anti-correlated`	69.9	151.0	18.1
`correlated`	70.0	144.5	17.6
`exact-ties`	48.1	105.7	11.6
`independent-positive`	70.8	148.2	17.3
`independent-signed`	70.4	142.4	17.2
`near-ties`	50.1	110.9	12.6

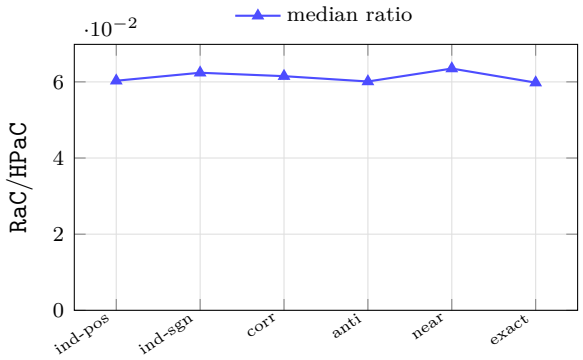

Figure 21: Stars by family: absolute times (left) and the paired ratio `RaC`/`HPaC` (right), between 0.060 and 0.064 in all six. The inversion is purely topological: no coefficient family produces it and none removes it.

n_nodes	Paired instances	Median rac/pac	IQR
100	60	5.778	0.454
200	60	3.675	0.192
500	60	1.883	0.120
1000	60	1.151	0.053
2000	60	0.524	0.034
5000	60	0.198	0.016
10000	60	0.100	0.005
20000	60	0.043	0.004
50000	60	0.015	0.001
100000	60	0.007	0.000

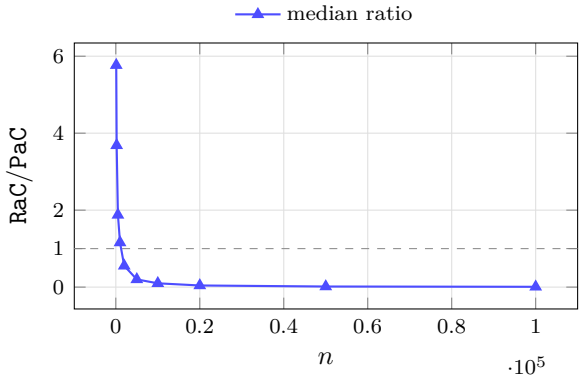

Figure 22: Stars, `RaC` against `PaC`: the comparison between the two algorithms that are *not* penalized by a candidate structure on a hub, and therefore the one that isolates the asymptotic difference. The crossover sits between $n = 1\,000$ and $n = 2\,000$; by $n = 10^5$ `RaC` is $130\times$ faster. Dashed line marks parity.

- **The star penalty is a property of the *schedule*, not of the algorithm.** `PaC` and `HPaC` implement the same algorithm; on stars the heap-based schedule is an order of magnitude slower than the scan-based one (Fig. 20), because maintaining candidate ratios is exactly what a hub makes expensive. A practitioner facing hub-shaped precedence graphs should reach for `RaC`, and failing that for the direct scan – not for the heap.

10 Single orientations

When the forest is known in advance to be a forest of in-trees or of out-trees, a specialized algorithm can exploit it. This section measures how much that knowledge is worth: it is a constant factor, not a change of regime, and the constant is remarkably stable across sizes, densities and coefficient families.

	CPU time (ms)			
n	`HPaC`	`DHPaC`	`HIPaC`	`RaC`
100	0.1	0.1	0.1	0.3
200	0.2	0.2	0.1	0.6
300	0.3	0.3	0.2	0.9
400	0.4	0.4	0.2	1.2
500	0.4	0.5	0.3	1.4
600	0.5	0.6	0.3	1.7
700	0.7	0.8	0.4	2.0
800	0.7	0.8	0.4	2.3
900	0.8	1.0	0.5	2.4
1 000	1.1	1.0	0.6	3.1
10 000	14.2	14.1	6.5	45.9
20 000	29.5	31.1	14.4	120.3
30 000	49.6	51.0	25.0	231.3
40 000	73.8	72.9	35.2	347.9
50 000	95.3	101.9	45.1	444.2
60 000	109.4	131.0	52.8	570.2
70 000	127.8	151.0	66.0	730.2
80 000	168.4	185.0	78.7	980.0
90 000	203.1	219.7	86.5	1 184.7
100 000	237.6	201.7	114.2	1 620.5

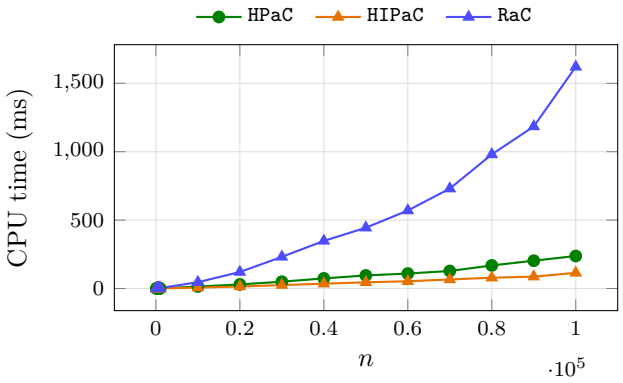

Figure 23: `in-forest` instances (240 per size): the specialized `HIPaC` against the general `HPaC` and against `RaC`. On a single orientation the minimal preceding set of every arc collapses to a singleton, so one heap value per vertex suffices and no closure-sum propagation is ever needed – that is where the constant factor comes from.

Note on the comparison with the first version of the manuscript. The specialized variants were reported there as 4–10× faster than the general algorithm, against 1.3–1.9× here. The direction is expected – the general `HPaC` is substantially faster than it was, see Section 11 – but the magnitude is only partly accounted for by the heap policy: comparing the two heap implementations directly on these instances gives 1.3–1.4×, not 3–4×. The remainder is attributable to the code being an independent rewrite rather than a refactoring, and to instances regenerated under different coefficient families. Absolute times across the two versions are therefore not comparable; only trends are.

10.1 What the oriented classes say

- **The specialization buys a stable constant, around** 1.5× **to** 2×**.** It does not change the growth rate, and it does not erode: the ratio at $n = 10^5$ is as good as at $n = 10^3$, and it is the same across densities and families. This is the cleanest, most predictable effect in the whole study.

n_nodes	Paired instances	Median hipac/hpac	IQR
100	240	0.749	0.445
200	240	0.662	0.474
300	240	0.628	0.554
400	240	0.615	0.553
500	240	0.619	0.578
600	240	0.580	0.573
700	240	0.565	0.597
800	240	0.603	0.622
900	240	0.582	0.589
1000	240	0.564	0.597
10000	240	0.521	0.682
20000	240	0.557	0.707
30000	240	0.570	0.781
40000	240	0.567	0.741
50000	240	0.575	0.765
60000	240	0.566	0.793
70000	240	0.589	0.819
80000	240	0.589	0.786
90000	240	0.574	0.789
100000	240	0.563	0.789

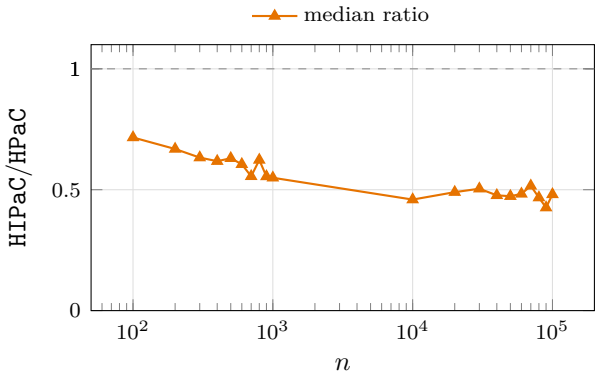

Figure 24: Paired ratio `HIPaC/HPaC` by size: between 0.52 and 0.75, with no erosion up to $n = 10^5$ – if anything the advantage grows mildly with n. Dashed line marks parity.

	CPU time (ms)			
n	`HPaC`	`DHPaC`	`HOPaC`	`RaC`
100	0.1	0.1	0.1	0.3
200	0.2	0.2	0.1	0.6
300	0.3	0.3	0.2	0.9
400	0.4	0.4	0.3	1.2
500	0.6	0.5	0.3	1.6
600	0.6	0.7	0.4	1.7
700	0.7	0.8	0.5	2.4
800	0.8	0.9	0.5	2.3
900	0.9	1.0	0.6	2.5
1 000	1.1	1.3	0.7	3.1
10 000	13.7	15.3	7.6	44.1
20 000	31.9	33.7	16.5	118.0
30 000	51.4	53.4	27.8	217.4
40 000	73.6	79.8	40.3	323.3
50 000	95.3	100.6	54.7	463.8
60 000	106.1	135.1	61.5	541.2
70 000	135.6	155.2	75.5	722.2
80 000	164.4	197.6	88.4	941.6
90 000	185.7	220.6	106.0	1 187.5
100 000	229.5	287.2	132.1	1 608.8

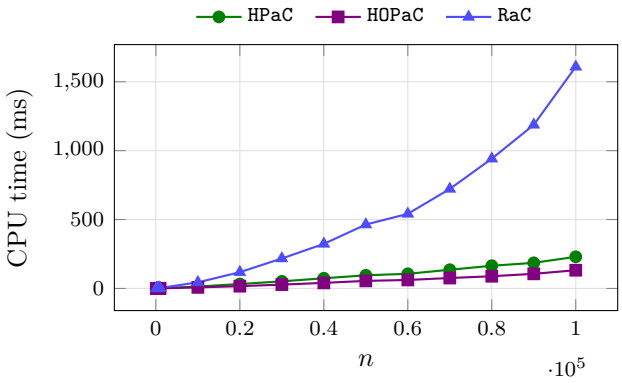

Figure 25: `out-forest` instances (240 per size): `HOPaC` against `HPaC` and `RaC`. The picture mirrors the in-forest one, as the symmetry of the two constructions predicts.

n_nodes	Paired instances	Median hopac/hpac	IQR
100	240	0.784	0.455
200	240	0.676	0.477
300	240	0.668	0.509
400	240	0.623	0.542
500	240	0.663	0.555
600	240	0.625	0.526
700	240	0.631	0.554
800	240	0.622	0.571
900	240	0.670	0.563
1000	240	0.619	0.592
10000	240	0.574	0.623
20000	240	0.585	0.648
30000	240	0.612	0.690
40000	240	0.619	0.698
50000	240	0.655	0.690
60000	240	0.624	0.742
70000	240	0.633	0.705
80000	240	0.636	0.669
90000	240	0.632	0.694
100000	240	0.632	0.682

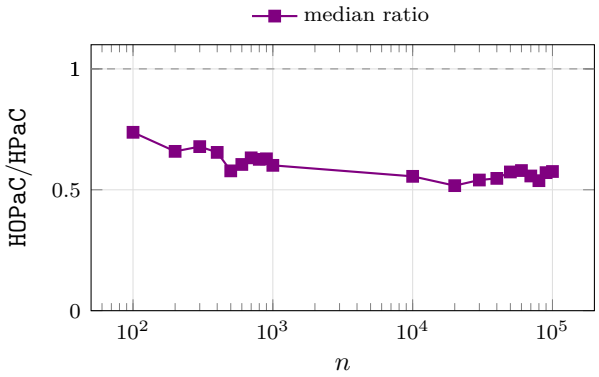

Figure 26: Paired ratio `HOPaC`/`HPaC` by size: between 0.57 and 0.78.

	median CPU time (ms)		
ϱ	`HPaC`	`HIPaC`	`RaC`
0.3	1.9	2.1	9.2
0.6	3.7	2.9	20.9
0.9	10.6	3.6	26.8
1.0	17.2	3.9	28.0

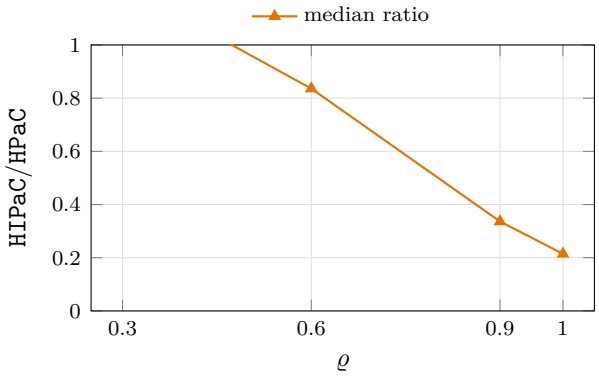

Figure 27: In-forests by density: absolute times (left) and the specialized-variant ratio (right). The gain of the specialization is stable in ϱ too, so it does not depend on how fragmented the forest is.

family	#inst	median `HIPaC`/`HPaC`	IQR
`anti-correlated`	800	0.689	0.705
`correlated`	800	0.697	0.741
`exact-ties`	800	0.671	0.617
`independent-positive`	800	0.676	0.710
`independent-signed`	800	0.652	0.659
`near-ties`	800	0.653	0.620

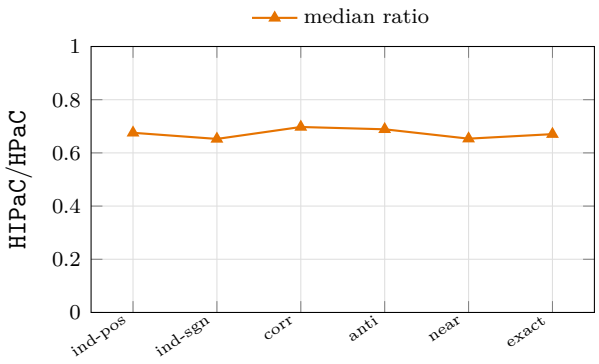

Figure 28: In-forests by family: the ratio table (left) and its plot (right). There is no family in which the advantage of the specialization disappears.

family	#inst	median RaC/HPaC	IQR
anti-correlated	800	3.007	2.165
correlated	800	3.066	2.300
exact-ties	800	3.235	2.427
independent-positive	800	3.025	2.274
independent-signed	800	3.039	2.262
near-ties	800	3.030	2.588

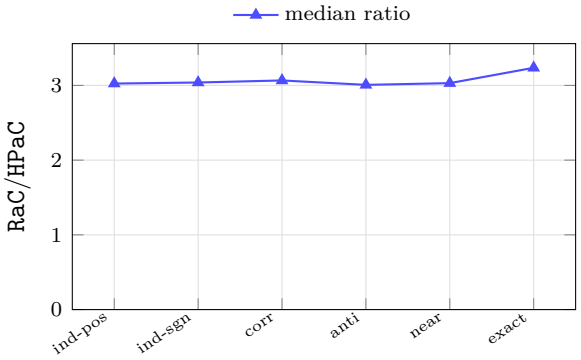

Figure 29: For completeness, RaC against the general HPaC on the same oriented instances, by family (out-forests behave the same). RaC loses ground as n grows here too, from about 2.8× at $n \leq 1\,000$ to 12.8× at $n = 10^5$.

- **It is smaller than the theory might suggest.** Both HIPaC and the general HPaC behave like $\mathcal{O}(n \log n)$ in practice on these instances, so what separates them is bookkeeping, not asymptotics: one heap value per vertex instead of two heaps plus closure-sum propagation.
- **Knowing the orientation is worth less than knowing the shape.** A factor 1.5–2 for a single orientation, against a factor 250 for a star. If one piece of structural information about the input can be exploited, the shape is the one that pays.

11 Implementation note: heap policy

The candidate heaps admit two implementations, indistinguishable in the asymptotic time bound and very different in practice.

- **Lazy deletion (push-only).** Every closure-sum change pushes a fresh entry; stale ones are dropped only when they surface at the top. Heap size is bounded by the number of touch operations, not by n. On a star this is pathological: every event touches the hub, each touch re-pushes one entry per incident arc, so the heap accumulates $\Theta(n^2)$ entries.
- **Periodic rebuild (used throughout this study).** Each heap is rebuilt from the live candidates whenever its size exceeds a constant multiple of the live candidate count: amortized $\mathcal{O}(\log n)$ per operation, worst-case time bound unchanged, memory $\mathcal{O}(n)$ on *every* input. This is what makes the implementation attain the space bound proved in the manuscript.

12 Comparison with parametric pseudoflow

BPPF evaluates minimum cuts at user-supplied parameter values, so the comparison must decide what to supply. We drive it in the setting most favourable to it, identical to the methodology of the manuscript's first version: one BPPF process per instance receives, in BPPF's native affine capacity format, the $k+1$ values that bracket all k breakpoints – so it is spared the search for the breakpoints, and one call certifies the whole parametric solution. Timing is BPPF's own internal solve timer (input parsing excluded) against HPaC's in-process time on the same instance; fixed-point precision 10^{-6}; all 2 400 instances with $n \leq 1\,000$.

Agreement and precision. The closures returned by the two methods agree at every probe on every instance, with one exception class: on **16 of the 2 400 instances** BPPF merges two consecutive closure layers whose thresholds differ by less than its fixed-point precision, and therefore does not

	lazy deletion		periodic rebuild	
n	ms	MiB	ms	MiB
1 000	139.0	15.6	64.1	3.9
2 000	628.7	51.5	233.1	4.0
5 000	5 440.9	388.0	1 422.1	6.4
10 000	27 020.7	1 541.2	6 396.5	8.2

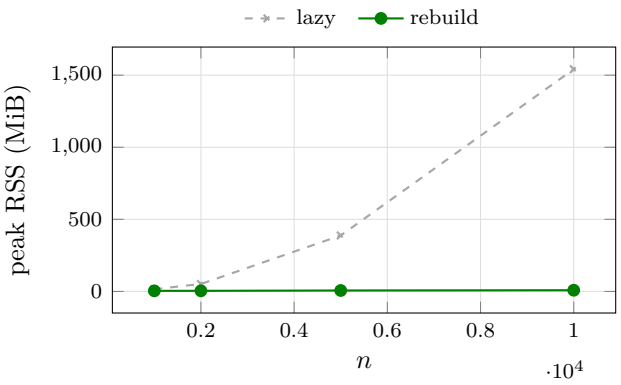

Figure 30: The two policies on `star-mixed` instances (`independent-positive`, seed 0, one repetition; single-algorithm invocations, the only regime in which peak RSS is attributable to one algorithm). The lazy heap's memory grows by a factor of 100 when n grows by 10 – quadratic, as predicted – reaching 1.5 GiB at $n = 10^4$ where the rebuild policy uses 8 MiB; extrapolating, the lazy policy exhausts an 8 GiB ceiling by $n \approx 20\,000$. The rebuild policy is also $\approx 4\times$ faster here (and $\approx 2\times$ on large random forests), with bit-identical output on every instance tested. For reference, `PaC`, which maintains no candidate structure, peaks at 5.6 MiB at $n = 10^4$. The practical lesson: on this problem the choice of data structure inside one algorithm is worth as much as the choice between algorithms.

		CPU time (ms)	
n	#inst	`HPaC`	`BPPF`
100	240	0.1	0.1
200	240	0.1	0.3
300	240	0.2	0.6
400	240	0.3	1.0
500	240	0.4	1.5
600	240	0.5	2.1
700	240	0.5	2.8
800	240	0.7	3.6
900	240	0.8	4.5
1 000	240	0.9	5.5

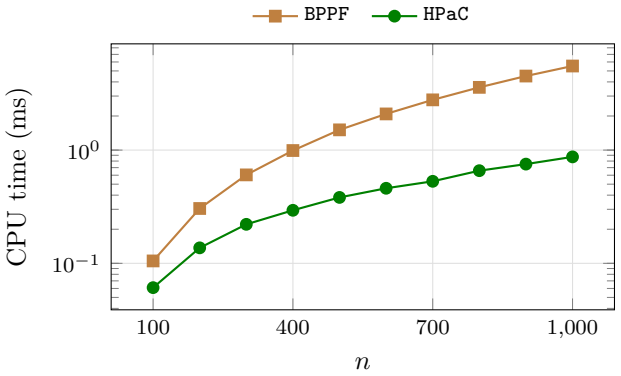

Figure 31: `HPaC` against `BPPF`. The median paired ratio grows from 1.5 at $n = 100$ to 4.5 at $n = 1\,000$ (overall median 3.3). Scope: forests with $n \leq 1\,000$, while `BPPF` solves a strictly more general problem, so this says nothing about `BPPF` outside this class. Note the times are sub-millisecond at the small end, which is why they are quoted here to one decimal in milliseconds and the ratio is computed per instance rather than from these columns.

recover the exact optimal layer sequence. All 16 belong to `near-ties` and `exact-ties`, the two families designed to place thresholds at exactly that distance – which is also why those families exist. Our algorithms are unaffected, comparing integers exactly.

Discarded variant. An earlier version of this comparison drove `BPPF` once per breakpoint, one process spawn per call. It is reported nowhere: the resulting ratio is dominated by process-creation overhead and says nothing about either algorithm.

13 Dispersion of the measurements

Every ratio reported above is a median over instances of a median over repetitions. This section reports the second-level dispersion – how much the repetitions of a single instance differ from each other – because that is the noise floor against which every claimed difference has to be read. It is expressed as the relative IQR (Section 3): the IQR over the repetitions of one instance, divided by that instance's median time.

	median relative IQR of the timing (%)		
n	`HPaC`	`DHPaC`	`RaC`
10 000	0.5	2.2	4.6
20 000	0.6	2.3	3.5
30 000	0.9	2.2	2.8
40 000	1.0	2.1	2.1
50 000	1.3	2.2	1.5
60 000	1.7	1.7	1.5
70 000	1.6	1.6	1.3
80 000	1.5	1.7	1.1
90 000	1.7	2.2	1.0
100 000	1.8	2.3	1.2

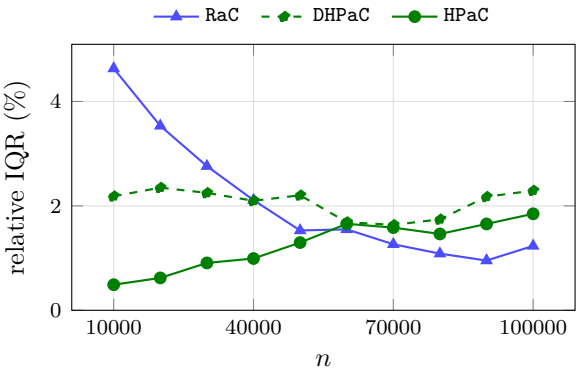

Figure 32: Median *relative* IQR of the timing (IQR over the repetitions of one instance, divided by that instance's median), campaign C. Below 1% for `HPaC` throughout, a few percent for `DHPaC` and `RaC`. This is the number that says how much of a reported difference is signal: every ratio in this report is far larger than the noise of the measurements it rests on. The single exception is the near-tie 0.91 against 1 at $n = 100$ in Section 8.1, which is reported as a near-tie and not as an advantage.

A Per-family detail at every size

The full cross-tabulation behind every by-family statement above: median CPU time per coefficient family and per size, for the two headline algorithms, on both random campaigns. The family ordering is stable across sizes – the two tie-stressing families are consistently the fastest for both algorithms, by a margin that widens with n (at $n = 10^5$, `near-ties` costs `HPaC` about 12% less than `independent-positive`) – and no family ever reverses the relative standing of the two algorithms. The plots show four representative families to stay readable; the tables give all six.

B Reproducing this report

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build --output-on-failure   # correctness gate
```

	median relative IQR of the timing (%)		
n	HPaC	HIPaC	RaC
100	8.7	4.2	12.3
200	4.5	3.1	4.4
300	3.1	2.9	5.4
400	3.7	3.2	4.7
500	2.5	3.2	3.4
600	3.0	2.4	3.1
700	3.0	2.3	2.7
800	3.4	2.4	3.0
900	2.7	2.3	3.1
1 000	3.9	2.2	4.1
10 000	4.5	2.4	3.6
20 000	3.6	2.9	2.7
30 000	2.6	3.2	2.3
40 000	2.6	3.3	1.6
50 000	2.6	3.5	1.2
60 000	2.6	3.4	1.1
70 000	2.5	3.7	0.9
80 000	2.7	3.2	0.7
90 000	2.6	3.0	0.9
100 000	2.5	2.6	1.1

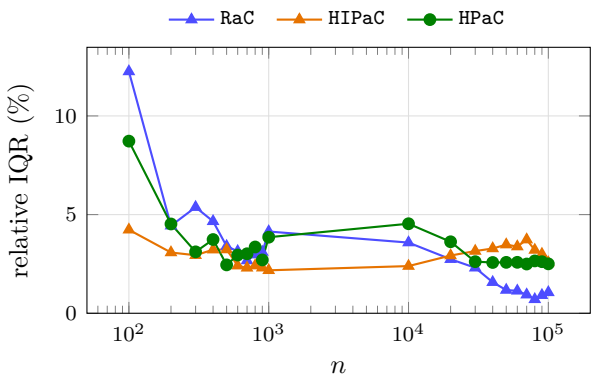

Figure 33: The same for in-forests: the specialized variant is as stable as the general algorithm, so its advantage is not an artefact of variance.

	median HPaC CPU time (ms) per family					
n	anti	corr	exact	ind-pos	ind-sgn	near
100	0.1	0.1	0.1	0.1	0.1	0.1
200	0.2	0.2	0.2	0.2	0.2	0.2
300	0.2	0.2	0.2	0.3	0.2	0.3
400	0.3	0.3	0.3	0.4	0.4	0.3
500	0.4	0.4	0.4	0.4	0.5	0.4
600	0.6	0.6	0.5	0.5	0.5	0.5
700	0.6	0.6	0.6	0.7	0.7	0.7
800	0.9	0.8	0.8	0.8	0.8	0.8
900	0.9	0.9	0.9	1.0	1.0	0.9
1 000	0.9	0.9	0.8	0.9	0.9	0.9

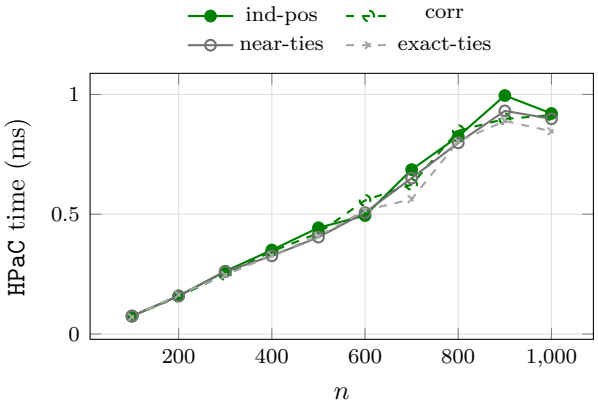

Figure 34: HPaC on mixed-forest instances, $n \leq 1\,000$.

	median RaC CPU time (ms) per family					
n	anti	corr	exact	ind-pos	ind-sgn	near
100	0.3	0.3	0.2	0.3	0.3	0.3
200	0.6	0.6	0.6	0.6	0.6	0.6
300	0.9	0.9	0.9	0.9	0.9	0.9
400	1.3	1.3	1.2	1.3	1.2	1.2
500	1.5	1.5	1.4	1.6	1.5	1.5
600	1.9	1.8	1.7	1.9	1.9	1.8
700	2.2	2.2	2.0	2.2	2.4	2.3
800	2.9	2.9	2.7	2.9	2.9	2.8
900	3.5	3.2	3.0	3.3	3.3	3.1
1 000	3.1	3.0	2.9	3.2	3.1	2.9

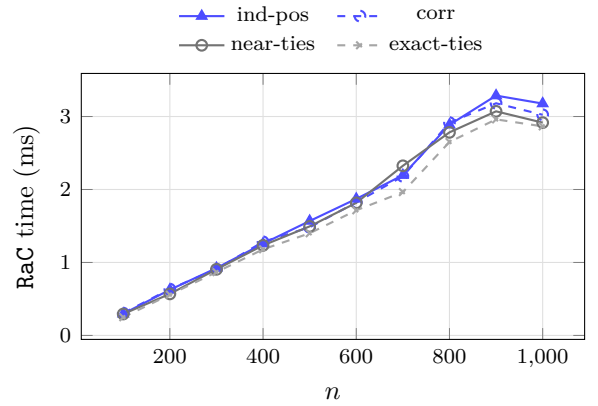

Figure 35: RaC on mixed-forest instances, $n \leq 1\,000$.

	median HPaC CPU time (ms) per family					
n	anti	corr	exact	ind-pos	ind-sgn	near
10 000	13.4	13.2	12.3	12.4	13.0	12.0
20 000	29.1	28.7	27.1	29.8	29.6	27.3
30 000	49.6	47.5	46.0	46.5	48.5	42.4
40 000	66.4	65.4	61.6	66.0	64.7	61.2
50 000	91.1	88.6	80.7	93.1	87.9	83.2
60 000	100.4	98.9	92.8	99.5	100.1	93.2
70 000	116.5	114.9	114.9	124.4	125.5	115.5
80 000	144.8	143.9	135.3	149.9	149.2	134.4
90 000	174.9	172.8	161.0	174.2	183.6	157.9
100 000	229.5	217.3	202.3	224.7	224.2	196.3

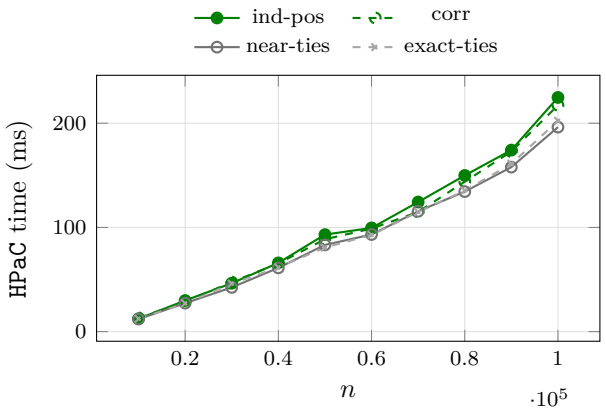

Figure 36: HPaC on mixed-forest instances, $n \geq 10\,000$.

	median RaC CPU time (ms) per family					
n	anti	corr	exact	ind-pos	ind-sgn	near
10 000	52.8	53.0	49.4	53.3	53.1	49.8
20 000	143.2	142.6	132.1	142.2	142.1	129.1
30 000	266.5	267.8	242.2	263.5	265.7	238.7
40 000	409.0	402.0	366.8	409.2	412.7	357.4
50 000	558.3	535.2	484.4	551.0	548.1	481.0
60 000	669.3	649.0	602.8	668.8	663.7	582.7
70 000	831.8	824.6	839.3	835.4	832.4	779.2
80 000	1 036.9	1 040.8	1 020.4	1 036.0	1 034.7	1 027.5
90 000	1 318.6	1 329.2	1 311.8	1 337.4	1 329.0	1 304.3
100 000	1 768.8	1 792.4	1 735.5	1 765.6	1 753.1	1 701.3

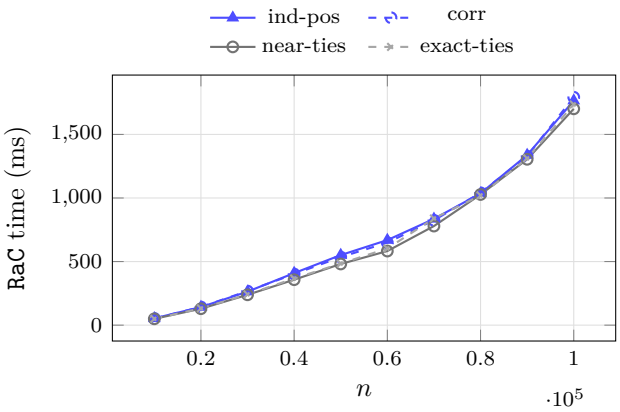

Figure 37: RaC on mixed-forest instances, $n \geq 10\,000$.

```
tools/run_night_sweep.sh                    # all campaigns, one command
tools/build_report.sh                       # aggregate + emit all tables
cd report && latexmk -pdf computational_report.tex
```

Code, generators, raw and processed data: https://github.com/fabiofurini/parametric-closure-forests, release `v0.3.0` (instance archives split under GitHub's asset size limit, with SHA-256 checksums). The pseudoflow baseline is the unmodified upstream implementation at https://github.com/hochbaumGroup/Bounded-precision-simple-parametric.git.